

03-排序

03-排序专题

一、本阶段目标

排序是 CSP / C++ 算法学习中非常基础、但非常重要的专题。

这一章的目标是：

- 理解排序在算法题中的作用
- 掌握 `sort` 的基本用法
- 理解 `cmp` 比较函数的本质
- 掌握数组、vector、pair、struct 的排序方式
- 能看懂并写出多关键字排序
- 知道排序和贪心、二分、模拟之间的关系

排序不是单纯地“把数排好”。

很多题真正的关键是：

先排序，改变数据顺序，让后面的处理变简单。

二、知识点总览

本专题包括：

- `sort` 基础用法
 - 普通数组排序
 - `vector` 排序
 - 降序排序
 - `cmp` 比较函数
 - `pair` 排序
 - `struct` 结构体排序
 - 多关键字排序
 - 排序与贪心
 - 排序与二分
 - 常见错误
-

三、sort 是什么？

`sort` 是 C++ `<algorithm>` 头文件中的排序函数。

头文件：

```
#include<algorithm>
```

作用：

把一段数据按照指定规则重新排列。

默认规则是：

从小到大排序。

四、sort 的基本原理

1. sort 做了什么？

比如原数组是：

5 2 4 1 3

执行排序后：

1 2 3 4 5

`sort` 的作用就是：

不断比较两个元素，判断谁应该排在前面，最终把整个序列排好。

2. sort 默认按什么排？

默认情况下：

```
sort(a, a+n);
```

等价于：

如果前一个数更小，就排在前面。

所以默认是升序。

3. sort 的时间复杂度

一般记为：

$O(n \log n)$

含义是：

排序比普通遍历慢，但比双重循环排序快很多。

普通双重循环排序通常是：

$O(n^2)$

当 `n` 很大时，`sort` 更适合比赛使用。

五、普通数组排序

1. 升序排序

```
sort(a, a+n);
```

含义：

排序范围是 `[a, a+n)`

也就是：

从 `a[0]` 排到 `a[n-1]`

注意：

右边界 `a+n` 不包含。

2. 示例代码

```
#include<iostream>
#include<algorithm>
using namespace std;

int main()
{
    int n;
    cin>>n;

    int a[100];

    for(int i=0;i<n;i++)
    {
        cin>>a[i];
    }

    sort(a,a+n);

    for(int i=0;i<n;i++)
    {
        cout<<a[i]<<" ";
    }

    return 0;
}
```

六、vector 排序

1. 升序排序

```
sort(a.begin(),a.end());
```

含义：

排序整个 vector。

2. 示例代码

```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;

int main()
{
    int n;
    cin>>n;

    vector<int> a(n);

    for(int i=0;i<n;i++)
    {
        cin>>a[i];
    }

    sort(a.begin(),a.end());

    for(int i=0;i<n;i++)
    {
        cout<<a[i]<<" ";
    }

    return 0;
}
```

七、降序排序

1. 使用 greater

```
sort(a.begin(),a.end(),greater<int>());
```

表示：

从大到小排序。

2. 示例代码

```
#include<iostream>
#include<vector>
```

```
#include<algorithm>
#include<functional>
using namespace std;

int main()
{
    vector<int> a={3,1,5,2,4};

    sort(a.begin(),a.end(),greater<int>());

    for(int i=0;i<a.size();i++)
    {
        cout<<a[i]<<" ";
    }

    return 0;
}
```

输出：

```
5 4 3 2 1
```

3. 使用 cmp 降序

```
bool cmp(int a,int b)
{
    return a>b;
}
```

排序：

```
sort(a.begin(),a.end(),cmp);
```

八、cmp 比较函数

1. cmp 是什么？

cmp 是比较函数。

它的作用是告诉 sort：

两个元素比较时，谁应该排在前面。

最重要的一句话：

`cmp(a,b)` 返回 `true`，表示 `a` 应该排在 `b` 前面。

2. 升序 `cmp`

```
bool cmp(int a,int b)
{
    return a<b;
}
```

含义：

如果 `a < b`，就让 `a` 排在 `b` 前面。

所以结果是升序。

3. 降序 `cmp`

```
bool cmp(int a,int b)
{
    return a>b;
}
```

含义：

如果 `a > b`，就让 `a` 排在 `b` 前面。

所以结果是降序。

4. 不要把 `cmp` 理解成“要不要交换”

很多初学者会把：

```
return a<b;
```

理解成：

a 比 b 小，所以交换。

这个理解是错的。

正确理解是：

return true 表示 a 应该在 b 前面。

5. 例子

```
bool cmp(int a,int b)
{
    return a>b;
}
```

比较 5 和 3：

5 > 3 为 true

说明：

5 应该排在 3 前面。

所以是降序。

九、pair 排序

1. pair 默认排序规则

对于：

```
pair<int,int>
```

默认排序规则是：

先按 first 从小到大排;
如果 first 相同, 再按 second 从小到大排。

2. 示例

```
vector<pair<int,int>> a;  
  
a.push_back({2,3});  
a.push_back({1,5});  
a.push_back({2,1});  
  
sort(a.begin(),a.end());
```

排序后:

```
(1,5)  
(2,1)  
(2,3)
```

3. 完整代码

```
#include<iostream>  
#include<vector>  
#include<algorithm>  
using namespace std;  
  
int main()  
{  
    vector<pair<int,int>> a;  
  
    a.push_back({2,3});  
    a.push_back({1,5});  
    a.push_back({2,1});  
  
    sort(a.begin(),a.end());  
  
    for(int i=0;i<a.size();i++)  
    {  
        cout<<a[i].first<<" "<<a[i].second<<"\n";  
    }  
}
```

```
    return 0;  
}
```

十、struct 结构体排序

1. 为什么需要结构体排序?

很多题中，一个对象不只有一个属性。

比如学生有：

```
姓名  
分数  
编号
```

这时不能只存一个数字。

可以用结构体：

```
struct student  
{  
    string name;  
    int score;  
};
```

2. 按分数从高到低排序

```
bool cmp(student a, student b)  
{  
    return a.score > b.score;  
}
```

含义：

```
如果 a 的分数比 b 高，就让 a 排在 b 前面。
```

3. 完整代码

```
#include<iostream>
#include<algorithm>
using namespace std;

struct student
{
    string name;
    int score;
};

bool cmp(student a,student b)
{
    return a.score>b.score;
}

int main()
{
    int n;
    cin>>n;

    student a[100];

    for(int i=0;i<n;i++)
    {
        cin>>a[i].name>>a[i].score;
    }

    sort(a,a+n,cmp);

    for(int i=0;i<n;i++)
    {
        cout<<a[i].name<<" "<<a[i].score<<"\n";
    }

    return 0;
}
```

十一、多关键字排序

1. 什么是多关键字排序？

多关键字排序就是：

先按第一条件排序；
如果第一条件相同，再按第二条件排序；

如果第二条件还相同，再按第三条件排序。

2. 例子

学生排序规则：

先按分数从高到低；
如果分数相同，按编号从小到大。

3. cmp 写法

```
bool cmp(student a, student b)
{
    if(a.score != b.score)
    {
        return a.score > b.score;
    }
    else
    {
        return a.id < b.id;
    }
}
```

4. 完整代码

```
#include<iostream>
#include<algorithm>
using namespace std;

struct student
{
    int id;
    int score;
};

bool cmp(student a, student b)
{
    if(a.score != b.score)
    {
        return a.score > b.score;
    }
```

```

    }
    else
    {
        return a.id<b.id;
    }
}

int main()
{
    int n;
    cin>>n;

    student a[100];

    for(int i=0;i<n;i++)
    {
        cin>>a[i].id>>a[i].score;
    }

    sort(a,a+n,cmp);

    for(int i=0;i<n;i++)
    {
        cout<<a[i].id<<" "<<a[i].score<<"\n";
    }

    return 0;
}

```

十二、cmp 的书写原则

1. cmp 必须返回 bool

```
bool cmp(...)
```

不能写成：

```
int cmp(...)
```

2. cmp 里面要表达“谁排前面”

例如：

```
return a.score>b.score;
```

翻译成：

如果 a 分数更高，a 排前面。

3. 不要写不稳定的比较

错误写法：

```
bool cmp(int a,int b)
{
    return a<=b;
}
```

原因：

当 a 和 b 相等时，cmp(a,b) 和 cmp(b,a) 都可能为 true，会破坏排序规则。

正确写法：

```
bool cmp(int a,int b)
{
    return a<b;
}
```

或者：

```
bool cmp(int a,int b)
{
    return a>b;
}
```

十三、排序和贪心的关系

很多贪心题都需要先排序。

因为排序可以让我们：

按照某种优先级处理数据。

比如：

- 按结束时间排序
- 按单位价值排序
- 按花费从小到大排序
- 按重量从小到大排序

例子：部分背包

每个物品有：

重量
价值
单位价值

策略：

按单位价值从高到低排序，优先拿单位价值高的。

cmp：

```
bool cmp(item a,item b)
{
    return a.unit>b.unit;
}
```

十四、排序和二分的关系

很多二分查找题必须先排序。

因为二分的前提是：

数据有序。

例如：

```
sort(a.begin(),a.end());
```

```
int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

如果不排序，二分查找就没有意义。

十五、排序和模拟的关系

有些模拟题也需要排序。

比如：

```
按时间顺序处理事件  
按编号顺序输出结果  
按坐标顺序扫描
```

排序后，模拟过程会更清晰。

十六、常见错误

1. 忘记加头文件

错误：

```
sort(a,a+n);
```

但没有：

```
#include<algorithm>
```

2. 普通数组排序范围写错

正确：

```
sort(a,a+n);
```

错误：

```
sort(a,a+n-1);
```


如果写 `a+n-1`，最后一个元素不会参与排序。

3. vector 排序范围写错

正确：

```
sort(a.begin(),a.end());
```

4. cmp 写反

想从高到低，却写成：

```
return a.score<b.score;
```

这会变成从低到高。

5. cmp 中用了 `<=` 或 `>=`

错误：

```
return a<=b;
```

正确：

```
return a<b;
```

6. 多关键字排序忘记处理相等情况

错误：

```
bool cmp(student a,student b)
{
    return a.score>b.score;
}
```

如果题目要求：

分数相同按编号排序

那就必须写第二关键字。

十七、本专题做过的题

题号	题目	类型	题面简述	对应知识点
P1177	【模板】排序	排序模板	给定若干整数，按从小到大输出	sort 基础
P1059	明明的随机数	排序 / 去重	输入若干随机数，去重后从小到大输出	sort + unique / set
P1068	分数线划定	结构体排序	根据报名者成绩划定分数线并输出入围者	struct + cmp
P1104	生日	多关键字排序	按生日先后排序，生日相同按输入顺序处理	struct + 多关键字
P2240	部分背包	贪心 / 排序	按单位价值排序后尽量装入背包	sort + cmp + 贪心

十八、本章最重要的结论

排序的核心不是代码：

```
sort(a.begin(), a.end());
```

而是思想：

通过重新排列数据，让后面的处理更容易。

cmp 的核心也不是背模板，而是理解：

cmp(a,b) 返回 true，表示 a 应该排在 b 前面。

看到排序题时，先问自己：

题目希望谁排在前面？

然后把这个规则写进 `cmp` 。